

Proyecto acustico | ACUS220
Benjamin Martinez - Katherine Zapata

Proyecto KM



Motivacion



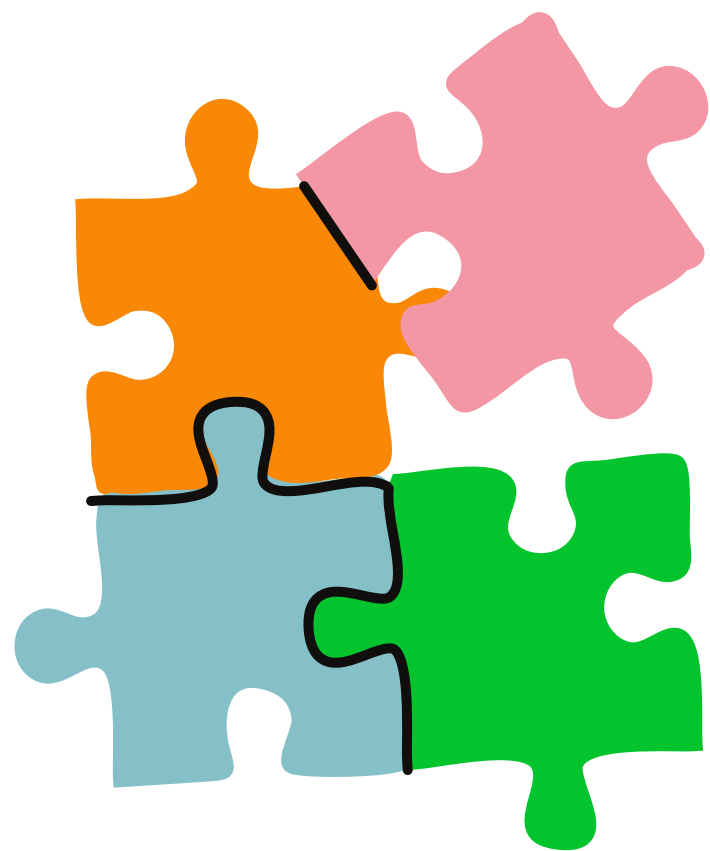
Problema que queremos abordar:

Las personas sordas o con pérdida auditiva profunda no tienen acceso directo a señales auditivas del entorno, lo que dificulta actividades cotidianas como identificar alertas, voces o sonidos relevantes.



Por qué nos interesa:

Nos motiva desarrollar herramientas tecnológicas que permitan traducir información sonora en formas accesibles, aprovechando nuestros conocimientos de programación y acústica.

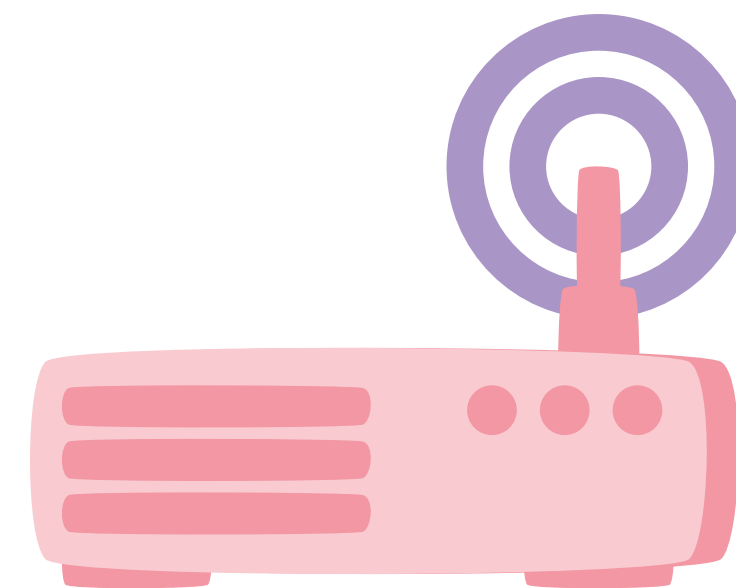


Beneficios esperados:

- Mayor autonomía para personas sordas.
- Traducción de señales auditivas a formatos visuales o analíticos.
- Herramientas que podrían integrarse en apps accesibles.

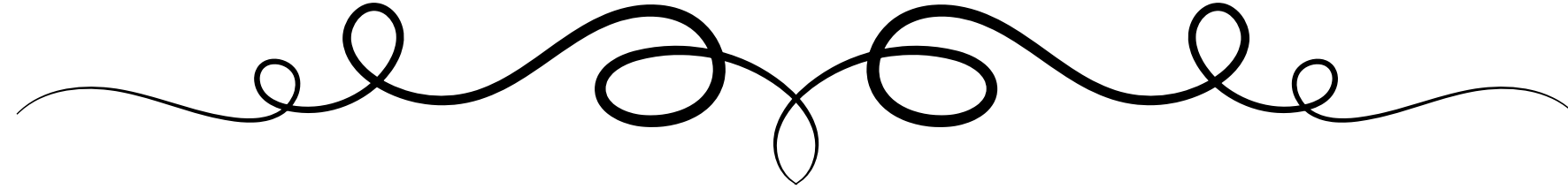
Qué queremos aprender:

- Procesamiento digital de señales (DSP).
- Manejo de audio real, filtrado, métricas acústicas.
- Implementar un pipeline reproducible y escalable.



Objetivo:

Desarrollar un sistema capaz de procesar señales de audio y traducirlas en información visual/computable para facilitar la interpretación de sonidos a personas sordas.

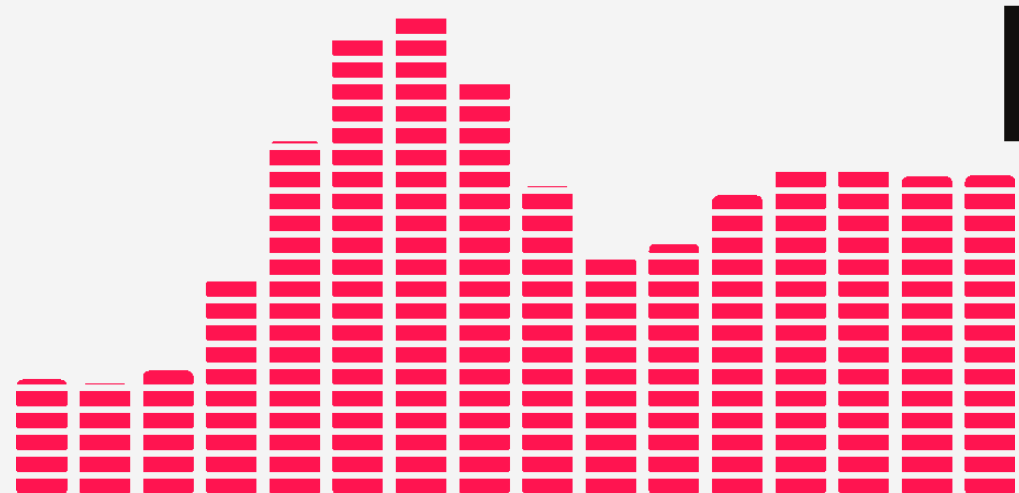


Objetivos específicos:

1. Implementar un pipeline de carga, limpieza y procesamiento del audio.
2. Extraer características relevantes (como energía, espectro, envolvente, etc.).
3. Clasificar o identificar patrones acústicos de interés.
4. Presentar los resultados de forma clara y fácilmente interpretable.

Definición de éxito:

El sistema procesa correctamente el audio, genera visualizaciones/valores útiles y permite reconocer patrones o eventos sonoros con precisión y claridad.



Descripción general del funcionamiento

Flujo principal:

Audio → Preprocesamiento → Features acústicos →
Análisis / Clasificación → Salida visual o numérica

Componentes principales:

- Entrada de audio (.wav)
- Filtros y normalización
- Cálculo de espectro / energía / envolvente
- Módulo analítico
- Visualización final

Tecnologías/librerías usadas:

Python, NumPy, SciPy, librosa, matplotlib, Jupyter Notebooks.



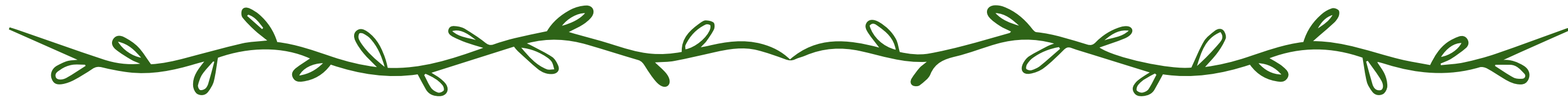
Datos y conceptos clave

Origen de los datos:

Archivos de audio capturados o provistos por el docente (señales WAV reales).

Tipos de datos:

- Señales continuas discretizadas
- Series numéricas
- Matrices espectrales



Variables que usamos:

- Señal de audio digital
- Frecuencia de muestreo
- Espectrograma
- Envolvente temporal
- Energía y RMS

Definiciones importantes:

- Feature acústico: magnitudes derivadas del audio (energía, espectro).
- Espectrograma: representación tiempo-frecuencia del sonido.
- Envolvente: curva que resume la amplitud del audio.
- RMS: nivel de energía perceptual del sonido.

Arquitectura

Módulos principales:

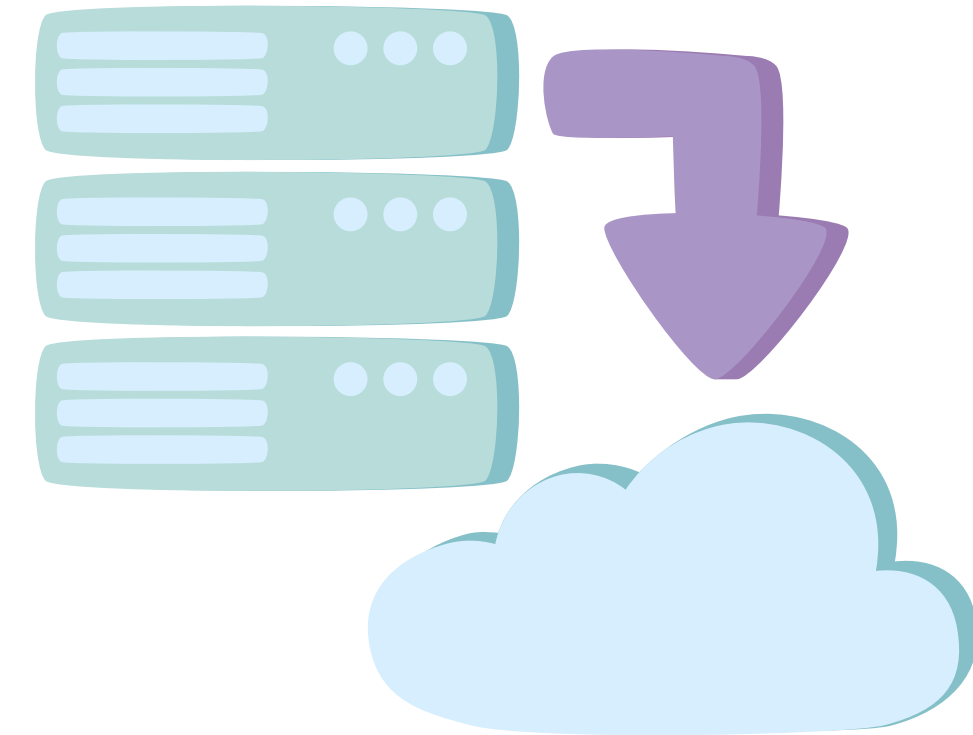
- Ingestión: cargar archivos WAV y metadatos.
- Preprocesamiento: normalización, filtrado y recorte.
- Extracción de features: espectrograma, RMS, envolvente, etc.
- Análisis: detección de eventos o patrones.
- Salida: gráficos, métricas y resultados numéricos.

Conexión entre módulos:

Cada notebook produce datos limpios que son usados por el siguiente, formando un pipeline reproducible.

Validaciones:

- Gráficos comparativos antes/después.
- Comprobación de niveles y espectros.
- Inspección visual de features.



Implementación

Archivos clave:

- preprocesamiento.ipynb
- features.ipynb
- analisis.ipynb
- Funciones auxiliares en utils.py

Funciones importantes:

- Carga y normalización de señales
- Cálculo de espectrograma
- Cálculo de RMS y envolvente
- Detección simple de eventos



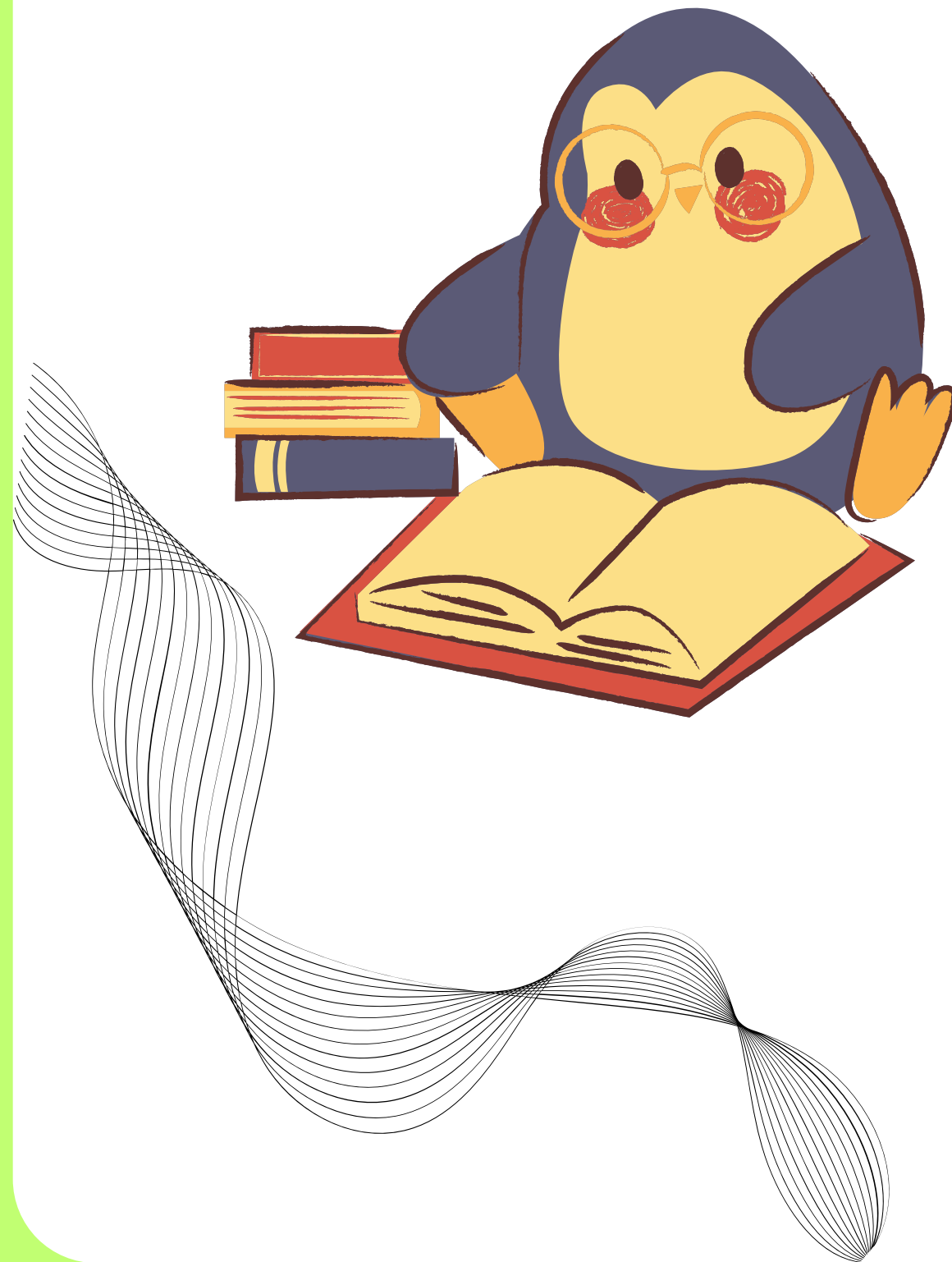
Librerías usadas:

NumPy, SciPy, librosa, matplotlib, soundfile.

Validación:

Probamos con audios distintos, verificamos niveles, revisamos gráficos y confirmamos que los features reaccionaran coherentemente ante sonidos fuertes o suaves.

Estado actual



Ya está funcionando:

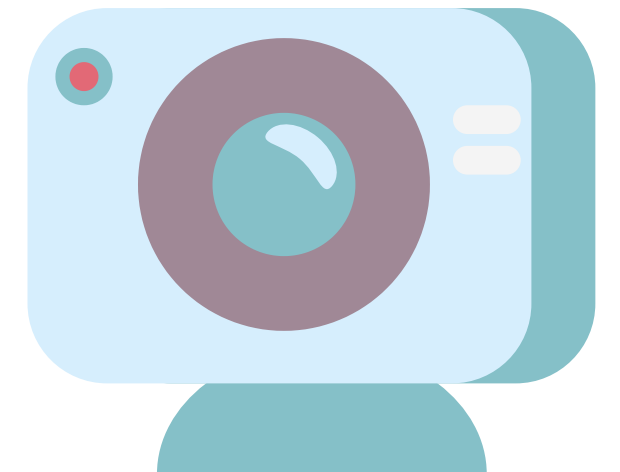
- Pipeline completo desde el audio crudo hasta los features procesados.
- Visualizaciones claras (espectrogramas, curvas RMS).
- Identificación básica de eventos.

Hallazgos:

- Ciertos sonidos tienen patrones espectrales fáciles de distinguir.
- La envolvente permite detectar eventos sin necesidad de modelos complejos.

Limitaciones actuales:

- Falta una clasificación más robusta.
- Falta optimización en tiempo real.





¡Muchas
gracias!

